endava

UNIVERSITATEA DIN BUCUREȘTI
VIRTUTE ET SAPIENTIA

PROGRAMARE ORIENTATA PE OBIECTE II

CURS 4

AGENDA CURS 4

1. Relatiile intre clase: mostenire versus compozitie
2. Clasa Object
3. Principiul imutabilitatii
4. Records
5. Lucrul cu siruri de caractere: clasele String, StringBuilder, StringBuffer

endava

MOSTENIRE VERSUS COMPOZITIE

Concept	Mostenire	Compozitie
Definire	O clasa extinde alta clasa	O clasa **contine** alta clasa
Relatie	"is-a" (este un tip de)	"has-a" (are un)
Sintaxa	`class A extends B`	Clasa A are un camp de tip B
Reutilizare	Preia metode si campuri din clasa baza	Apeleaza metode ale unui obiect
Flexibilitate	Scazuta (mostenire rigida)	Ridicata (poti schimba componenta)
Encapsulare	Mai slaba (expune internul clasei baza)	Mai buna (obiectul este separat)
Multipla?	Nu (Java nu permite mostenire multipla)	Da (poate avea mai multe componente)
Favorit in OOP	Mai putin recomandata in general	Recomandata pentru modularitate

endava

CLASA OBJECT

Metoda	Descriere scurta
`equals(Object obj)`	Compara doua obiecte pentru egalitate logica
`hashCode()`	Returneaza un cod hash asociat obiectului (corelat cu `equals`)
`toString()`	Returneaza un `String` care reprezinta obiectul
`getClass()`	Returneaza obiectul `Class` al instantei (metadate despre clasa)
`clone()`	Creeaza o copie (shallow) a obiectului (necesita implementarea `Cloneable`)
`finalize()`	Este apelata inainte ca obiectul sa fie colectat de garbage collector
`notify()`	Trezeste un thread aflat in asteptare pe monitorul obiectului
`notifyAll()`	Trezeste toate thread-urile in asteptare pe monitorul obiectului
`wait()`	Pune thread-ul curent in asteptare pana cand este notificat
`wait(long timeout)`	Asteapta o perioada determinata de timp sau pana cand este notificat
`wait(long timeout, int nanos)`	Asteapta cu precizie de nanosecunde

endava

PRINCIPIUL IMUTABILITATII

- Imutabilitatea se refera la obiecte a caror stare nu poate fi modificata dupa ce au fost create. In Java, clasele imutabile sunt utile pentru scrierea de cod sigur, mai ales in contexte concurente (multithreading).
- Caracteristici ale unei clase imutabile:
  1. Clasa este declarata final, pentru a preveni mostenirea.
  2. Atributele sunt private si final, astfel incat valorile lor nu pot fi schimbate dupa initializare.
  3. Nu exista setteri – doar metode getter.
  4. Campurile de tip obiect sunt copiate (deep copy) sau returnate ca o copie, pentru a evita modificarea lor din exterior.
  5. Valoarea este setata doar in constructor.
- Beneficii:
  - Usurinta in debug si testare.
  - Siguranta in contexte concurente (nu necesita sincronizare).
  - Comportament previzibil si mai putine erori legate de stare partajata.

PRINCIPIUL IMUTABILITATII

```java
public final class Persoana {
    private final String nume;
    private final int varsta;

    public Persoana(String nume, int varsta) {
        this.nume = nume;
        this.varsta = varsta;
    }

    public String getNume() {
        return nume;
    }

    public int getVarsta() {
        return varsta;
    }
}
```


endava

RECORDS

- **record** este un tip special de clasa introdus in Java 14 (preview) si stabil in Java 16
- Records sunt folosite pentru a defini obiecte imutabile de tip data carrier (obiecte care doar transporta date)
- Caracteristici principale:
  - Sintaxa simplificata pentru clase cu doar campuri, constructor si metode get.
  - Campurile sunt implicit private final.
  - Clasa este implicit final – nu poate fi extinsa.
  - Se genereaza automat: constructor, get-uri, equals(), hashCode() si toString().
- Limitari:
  - Nu poti avea campuri mutabile fara a pierde imutabilitatea.
  - Nu poate extinde alte clase
  - Poate implementa interfete.
  - Deep copy trebuie facut manual pentru obiecte mutabile.
- Avantaje:
  - Cod mai scurt si mai clar.
  - Ideal pentru DTO-uri, modele simple, key-value pairs etc.
  - Promoveaza imutabilitatea.

endava

RECORDS

```java
public record Person(String name, int age) {}
```

```java
public final class Person {
    private final String name;
    private final int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String name() { return name; }
    public int age() { return age; }

    // equals(), hashCode(), toString() sunt generate automat
}
```


endava

RECORDS

Caracteristica	`record`	Clasa normala
Sintaxa	Scurta, concisa	Verbosa (mult boilerplate)
Imutabilitate	Implicita (`final` + campuri `final`)	Trebuie implementata manual
Constructor	Generat automat	Scris manual
Getteri	Generati automat (fara prefix `get`)	Scrisi manual
`equals()` / `hashCode()`	Generati automat	Trebuie suprascrisi manual
`toString()`	Generat automat	Trebuie suprascris manual
Extindere clasa	Nu este permisa (`record` este `final`)	Posibila
Implementare interfete	Da	Da
Scop	Transport de date (DTO, modele simple)	Orice (logica, mostenire, structuri complexe)

endava

CLASA STRING

1. String este o clasa finala din pachetul **java.lang**.
2. Reprezinta o secventa de caractere.
3. Obiectele de tip String sunt imutabile – o data create, nu pot fi modificate
4. Valorile de tip String sunt stocate in String Pool.
5. String Pool este o zona speciala a memoriei heap din JVM. Scopul ei este de a optimiza memoria prin reutilizarea obiectelor String care au aceeasi valoare.
6. Daca doua String-uri au acelasi continut si sunt create ca literale, **JVM foloseste acelasi obiect pentru a le stoca.**

```
String s1 = "hello";
String s2 = "hello";

System.out.println(s1 == s2); // true – ambele referinteaza acelasi obiect din pool

String s3 = new String("hello");
System.out.println(s1 == s3); // false – s3 este un obiect nou

String s4 = s3.intern();
System.out.println(s1 == s4); // true – s4 foloseste obiectul din pool
```

CLASA STRING

Metoda	Descriere
`length()`	Returneaza lungimea stringului
`charAt(int index)`	Returneaza caracterul de la o anumita pozitie
`substring(int begin, int end)`	Returneaza un sub-string din stringul original
`equals(Object obj)`	Compara doua stringuri pentru egalitate
`equalsIgnoreCase()`	Compara stringuri ignorand diferenta dintre litere mari/mici
`compareTo(String another)`	Compara lexicografic doua stringuri
`toLowerCase()` / `toUpperCase()`	Transforma toate caracterele in litere mici / mari
`trim()`	Elimina spatiile de la inceput si sfarsit
`split(String regex)`	Imparte stringul in functie de o expresie regulata
`replace()`	Inlocuieste caractere sau secvente in string
`contains()`	Verifica daca stringul contine o anumita secventa
`startsWith()` / `endsWith()`	Verifica inceputul / sfarsitul stringului

Official API docs: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/package-summary.html

endava

LUCRUL CU SIRURI DE CARACTERE

Caracteristica	`String`	`StringBuilder`	`StringBuffer`
Mutabilitate	Imutabil (nu se poate modifica)	Mutabil (poate fi modificat)	Mutabil (poate fi modificat)
Thread-safe	Da (prin imutabilitate)	Nu	Da (metode sincronizate)
Performanta	Mai lent pentru operatii multiple	Cel mai rapid (fara sincronizare)	Mai lent decat `StringBuilder`
Utilizare recomandata	Cand continutul nu se schimba	Cand ai operatii intense pe stringuri intr-un singur thread	Cand ai operatii intense pe stringuri in mediu multithread
Sincronizare	Nu este necesara	Nu	Da
Clasa finala?	Da	Nu	Nu
Creare obiect	`String s = "abc";`	`StringBuilder sb = new StringBuilder();`	`StringBuffer sb = new StringBuffer();`

endava

Q&A

THANK YOU

DANIELA SPILCĂ